PROJECT REPORT

Student Academic Assistant Chatbot (CLI-Based)

Course: Fundamentals in AI and ML (CSA2001)
University: VIT Bhopal
Name: Tanishq Srivastava
Registration Number: 25BAS10025
Branch: B.Tech. in Aerospace Engineering
School: SMEC

1. Abstract

In today's academic environment, students frequently encounter challenges in accessing quick and relevant information regarding coursework, deadlines, and study strategies. This project presents a **Student Academic Assistant Chatbot**, a rule-based system developed using Python that interacts with users through a command-line interface.

The chatbot provides responses to common academic queries such as course details, deadlines, and study tips. The system demonstrates how basic programming concepts like conditional statements, loops, and data structures can be used to solve real-world problems.

The project highlights simplicity, usability, and practical application of knowledge, making it an effective example of problem-solving using computational thinking.

2. Introduction

With the rapid growth of digital education systems, students are exposed to a large amount of academic information. However, accessing this information quickly and efficiently remains a challenge.

Students often:

- Forget important deadlines
- Feel confused about course structures
- Lack proper study guidance

Traditional methods of obtaining information, such as browsing portals or asking peers, are time-consuming and inefficient.

To address this issue, this project introduces a **rule-based chatbot** that acts as a virtual assistant for students. The chatbot provides instant responses and simplifies access to academic information.

3. Background Study

Chatbots are computer programs designed to simulate human conversation. They are widely used in:

- Customer service
- Education systems
- Virtual assistants

There are two main types of chatbots:

1. Rule-based chatbots
2. AI-based chatbots

This project focuses on **rule-based chatbots**, which:

- Use predefined rules
- Match keywords
- Provide fixed responses

Although simpler than AI-based systems, they are:

- Easier to build
- Efficient for specific tasks
- Highly reliable for structured queries

4. Problem Statement

Students face several challenges in their academic journey:

- Difficulty remembering deadlines
- Lack of quick access to course information
- Poor study planning

These issues result in:

- Missed submissions
- Reduced productivity
- Increased stress

There is a need for a simple system that:

- Provides quick responses
- Is easy to use
- Requires minimal setup

5. Objectives

The objectives of this project are:

- To design a chatbot that assists students academically
- To provide quick and accurate responses
- To implement a rule-based system using Python
- To demonstrate real-world application of programming concepts
- To improve user interaction through simple design

6. Scope of the Project

The scope of this project includes:

- Command-line interaction
- Predefined responses
- Keyword-based matching

Limitations:

- No natural language understanding
- Limited to predefined queries
- No real-time data integration

Future improvements can extend the system into:

- AI-based chatbot
- Web or mobile application
- Integration with databases

7. System Requirements

Hardware Requirements:

- Basic computer system
- Minimum 4GB RAM

Software Requirements:

- Python (version 3.x)
- VS Code or any code editor
- Command-line interface

8. System Design

8.1 Architecture

The system follows a simple architecture:

User Input → Processing → Response Generation → Output

8.2 Flowchart (Explain in report)

1. Start
2. Take user input
3. Convert input to lowercase
4. Match keywords
5. Generate response
6. Display output
7. Repeat until exit

9. Methodology

The chatbot is implemented using a structured approach:

Step 1: Input Handling

User enters text via terminal

Step 2: Preprocessing

Input is converted to lowercase

Step 3: Keyword Matching

Input is compared with predefined keys

Step 4: Response Selection

Matching response is selected

Step 5: Output Display

Response is printed

Step 6: Loop Execution

Process continues until exit

10. Implementation

The project is implemented using Python programming language.

Key Components:

- `main.py` → handles interaction
- `responses.py` → stores logic

Modules Used:

- random → for varied responses

11. Features of the System

- Greeting system
- Personalized interaction
- Course information
- Deadline reminders
- Study tips
- Motivation messages
- Exit functionality
- Error handling

12. Code Explanation

Input Handling:

The chatbot continuously takes user input using a loop.

Keyword Matching:

Dictionary is used to map keywords to responses.

Response Generation:

Random module selects responses for variation.

Exit Condition:

Program stops when user types "exit".

13. Results and Output

The chatbot successfully:

- Processes user input
- Matches keywords
- Provides relevant responses

Sample Output:

(see last page for references)

14. Testing

The system was tested with:

- Valid inputs
- Invalid inputs
- Edge cases

Results:

- Correct responses for known queries
- Default responses for unknown queries

15. Challenges Faced

- Designing meaningful responses
- Handling unmatched inputs
- Maintaining simplicity

16. Advantages

- Easy to use
- Fast response
- Lightweight system
- No external dependencies

17. Limitations

- No AI capability
- Limited understanding
- Static responses

18. Future Enhancements

- NLP integration
- Voice interaction
- GUI development
- Database connectivity

19. Applications

- Educational platforms
- Student support systems
- Help desks

20. Learning Outcomes

- Learned Python programming
- Understood chatbot logic
- Improved problem-solving skills
- Learned modular coding

21. Conclusion

This project successfully demonstrates how a simple rule-based chatbot can solve real-world academic problems. It highlights the importance of structured programming and logical thinking in building practical solutions.

22. References

- Python Documentation
- Online programming tutorials
- Academic resources

OUR OUTPUTS

responses.py - C:/Users/SURFACE/Desktop/responses.py (3.14.0)

File Edit Format Run Options Window Help

```python
# Predefined responses
responses = {
    "hi": ["Hello!", "Hi there!", "Hey!"],
    "hello": ["Hello!", "Greetings!", "Hi!"],
    "courses": [
        "You have courses like CSE, CSA, Mathematics, Physics, etc.",
        "Your subjects include programming, mathematics, and core engineering courses."
    ],
    "deadline": [
        "Your project deadline is 31st March. Stay focused!",
        "Make sure to submit before 31st March to avoid penalties."
    ],
    "study": [
        "Study daily, revise concepts, and practice coding.",
        "Break your study into small sessions for better focus."
    ],
    "motivation": [
        "Stay consistent, success follows discipline ",
        "Small progress every day leads to big success!"
    ],
    "help": [
        "You can ask me about courses, deadlines, or study tips.",
        "Try asking: courses, deadline, or study."
    ]
}

# Default responses if nothing matches
default_responses = [
    "I didn't understand that.",
    "Can you rephrase your question?",
    "I'm still learning, try asking something else!"
]
```

Ln: 20 Col: 0

33°C Mostly cloudy | Search | ENG IN | 100% | 7:39 PM 3/31/2026

responses.py - C:/Users/SURFACE/Desktop/responses.py (3.14.0)

File Edit Format Run Options Window Help

```python
        "Make sure to submit before 31st March to avoid penalties."
    ],
    "study": [
        "Study daily, revise concepts, and practice coding.",
        "Break your study into small sessions for better focus."
    ],
    "motivation": [
        "Stay consistent, success follows discipline ",
        "Small progress every day leads to big success!"
    ],
    "help": [
        "You can ask me about courses, deadlines, or study tips.",
        "Try asking: courses, deadline, or study."
    ]
}

# Default responses if nothing matches
default_responses = [
    "I didn't understand that.",
    "Can you rephrase your question?",
    "I'm still learning, try asking something else!"
]


def get_response(user_input):
    user_input = user_input.lower()

    for key in responses:
        if key in user_input:
            return random.choice(responses[key])

    return random.choice(default_responses)
```

Ln: 24 Col: 0

33°C Mostly cloudy | Search | ENG IN | 100% | 7:40 PM 3/31/2026

main.py - C:/Users/SURFACE/Desktop/main.py (3.14.0)

File Edit Format Run Options Window Help

```python
from responses import get_response

def chatbot():
    print("Chatbot: Hello! What is your name?")
    name = input("You: ")

    print(f"Chatbot: Nice to meet you, {name}! Type 'exit' to quit.")

    while True:
        user_input = input("You: ")

        if user_input.lower() == "exit":
            print(f"Chatbot: Goodbye {name}! 👋")
            break

        response = get_response(user_input)
        print("Chatbot:", response)


chatbot()
```

Ln: 12 Col: 0

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=================== RESTART: C:/Users/SURFACE/Desktop/main.py ==================
Chatbot: Hello! What is your name?
You: Tanishq Srivastava
Chatbot: Nice to meet you, Tanishq Srivastava! Type 'exit' to quit.
You: deadline
Chatbot: Make sure to submit before 31st March to avoid penalties.
You: courses
Chatbot: Your subjects include programming, mathematics, and core engineering courses.
You: help
Chatbot: You can ask me about courses, deadlines, or study tips.
You: exit
Chatbot: Goodbye Tanishq Srivastava! 👋
>>>
```

Ln: 16 Col: 0